Department of Electrical Engineering

Indian Institute of Technology, Kharagpur

Algorithms, AI and ML Laboratory (EE22202)

Spring, 2025-26

Report 6: Artificial Neural Networks

Name: Arijit Dey

Roll No: 24IE10001

Artificial Neural Networks

1 MLPBackpropagation Class Definition

This section defines the core `MLPBackpropagation` class, which implements a Multi-Layer Perceptron (MLP) capable of learning through the backpropagation algorithm. The class includes methods for forward propagation, backward propagation (weight and bias updates), prediction, and accuracy calculation.

MLPBackpropagation Class

```python
class MLPBackpropagation:
    def __init__(self, n_in: int, n_hidden: list, n_out: int):
        self.layers = [n_in] + n_hidden + [n_out]
        self.weights = [np.random.randn(self.layers[i], self.layers[i+1]) * 0.1
                        for i in range(len(self.layers) - 1)]
        self.biases = [np.zeros((1, self.layers[i+1])) for i in range(len(self.layers) - 1)]
        # ADDITIONAL SETUP CODE...
```

1.1 The Signmod Activation Function

We use the sigmoid activation function for all the neurons (except at the input layer). It is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

while its derivative is

$$\nabla_x \sigma(x) = \frac{e^{-x}}{(1 - e^{-x})^2}$$

which can be simplified to

$$\nabla_x \sigma(x) = \sigma(x)(1 - \sigma(x))$$

Sigmod Activation Function

```python
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(o):
    return o * (1 - o)
```

1.2 Feedforwarding and Backpropagation

The `forward()` function takes a input vector x and traverses it through the network via the hidden layers ultimately to the output layer. At each layer l, it computes the following

$$x_{l+1} = \sigma(x_l \cdot w_l + b_l)$$

The `backward()` function is responsible for actually training the network.

1. The function starts at the output layer and calculates ε, the error between the model's predicted value and the actual output from the dataset.

$$\varepsilon = t - o$$

2. Next, it calculates the δ term for the output layer as

$$\delta_o = \varepsilon \frac{\delta \sigma}{\delta o}$$

3. This step is repeated for each layer l, starting at the output layer to the input layer calculating its error and δ as

$$\varepsilon = \delta_{l+1} \cdot w_l^T$$
$$\delta_l = \varepsilon \frac{\delta \sigma}{\delta x_{l+1}}$$

Lastly, we update the weights and biases using the weights-update rule

$$w_{l,\text{ new}} = w_l - \eta(x_{l+1} \cdot \delta_l)$$
$$b_{l,\text{ new}} = b_l - \eta(\delta_l)$$

Feedforward and Backpropagation Functions

```python
    def forward(self, x):
        activations = [x]
        for i in range(len(self.weights)):
            net = np.dot(activations[-1], self.weights[i]) + self.biases[i]
            out = sigmoid(net)
            activations.append(out)
        return activations

    def backward(self, x, t, current_eta):
        activations = self.forward(x)
        o = activations[-1]
        error_output = t - o
        delta = [sigmoid_derivative(o) * error_output]

        for i in range(len(self.weights) - 1, 0, -1):
            error_hidden = np.dot(delta[-1], self.weights[i].T)
            d_hidden = sigmoid_derivative(activations[i]) * error_hidden
            delta.append(d_hidden)

        delta.reverse()
        for i in range(len(self.weights)):
            self.weights[i] += current_eta * np.dot(activations[i].T, delta[i])
            self.biases[i] += current_eta * delta[i]
```

1.3 The Confusion Matrix

The confusion matrix is a $N \times X$ matrix which compares the actual values from the input dataset against the predicted values generated by your model. For a dataset with 2 classes, it is a 2x2 matrix.

	Class 1	**Class 2**
Class 1	Class 1 correctly classified as class 1	Class 1 wrongly classified as class 2
Class 2	Class 2 wrongly classified as class 1	Class 2 correctly classified as class 2

Calculating Confusion Matrix

```python
    def _get_confusion_matrix(self, y_true, y_pred):
        num_classes = self.layers[-1]
        cm = np.zeros((num_classes, num_classes), dtype=int)
        for i in range(num_classes):
            for j in range(num_classes):
                cm[i, j] = np.sum((y_true == i) & (y_pred == j))
        return cm
```

1.4 The `.fit()` function

The `fit` method orchestrates the training process of the MLP. It intentionally has a long list of parameters to flexibly allow us train our model against various training sets with various settings.

Calculating Confusion Matrix

```python
     def fit(self, X_train, y_train, X_val=None, y_val=None, X_test=None, y_test=None,
            eta=0.1, alpha=1.0, LR_decay_freq=10**9, maxIter=1000, maxErr=-1.0,
            percentOverfit=100.0, shuffle=True, cms_freq=None):

        # ...INITIAL SETUP CODE OMITTED

        for iteration in range(1, maxIter + 1):
            indices = np.arange(len(X_train))
            if shuffle:
                np.random.shuffle(indices)

            for i in indices:
                self.backward(X_train[i:i+1], y_train[i:i+1], curr_eta)

            if iteration % LR_decay_freq == 0:
                curr_eta *= alpha

            # -- COST, ACCURACY & CONFUSION MATRIX STORING CODE OMITTED --

            if (100.0 - s_acc) <= maxErr or (t_acc - v_acc) >= percentOverfit:
                break
```

2 XOR and XNOR Operations

We apply the `MLPBackpropagation` model to learn the XOR and XNOR logic problems, which are classic examples of non-linearly separable data. It showcases the model's ability to learn simple Boolean functions. The `learn_xor` function encapsulates the training and history extraction for these tasks.

XOR and XNOR Learning Code

```python
X_xor = np.array([[0,0], [0,1], [1,0], [1,1]])
y_xor = np.array([[0], [1], [1], [0]])

X_xnor = np.array([[0,0], [0,1], [1,0], [1,1]])
y_xnor = np.array([[1], [0], [0], [1]])

```

```
model_xor = MLPBackpropagation(2, [], 1)
model_xor.fit(X_xor, y_xor, X_val=X_xor, y_val=y_xor, X_test=X_xor, y_test=y_xor,
              eta=0.1, maxIter=1000, maxErr=-1.0, percentOverfit=100.0, shuffle=False)
```

XOR Equation Output

```
Final XOR Equation: y = sigmoid(-0.0262*x1 + -0.0142*x2 + 0.0141)
```

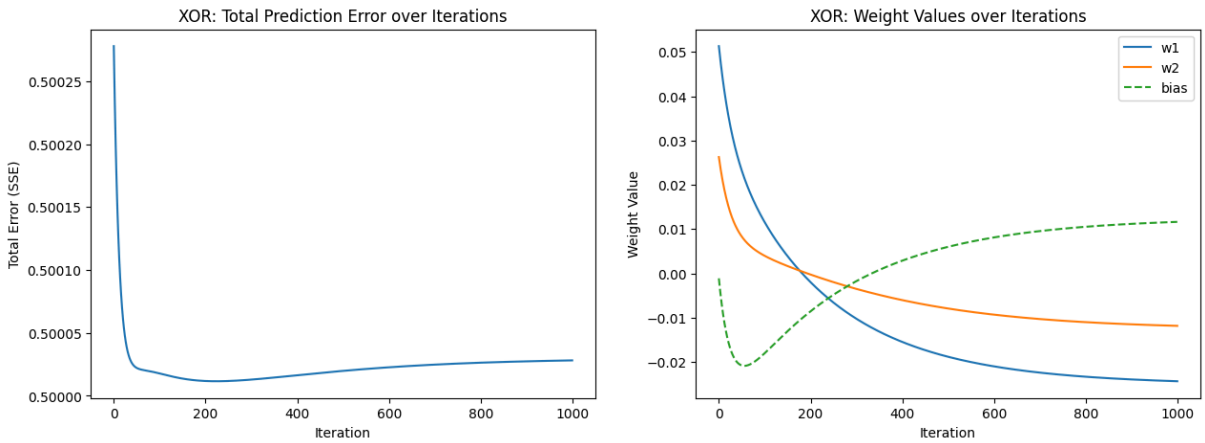

Figure 1: XOR Training: Total Prediction Error and Weight Values over Iterations.

3 Training on shuffled data

Effect of Example Order

```
model_xor_sf = MLPBackpropagation(2, [], 1)
model_xor_sf.fit(X_xor, y_xor, X_val=X_xor, y_val=y_xor, X_test=X_xor, y_test=y_xor,
              eta=0.1, maxIter=1000, maxErr=-1.0, percentOverfit=100.0, shuffle=True)
```

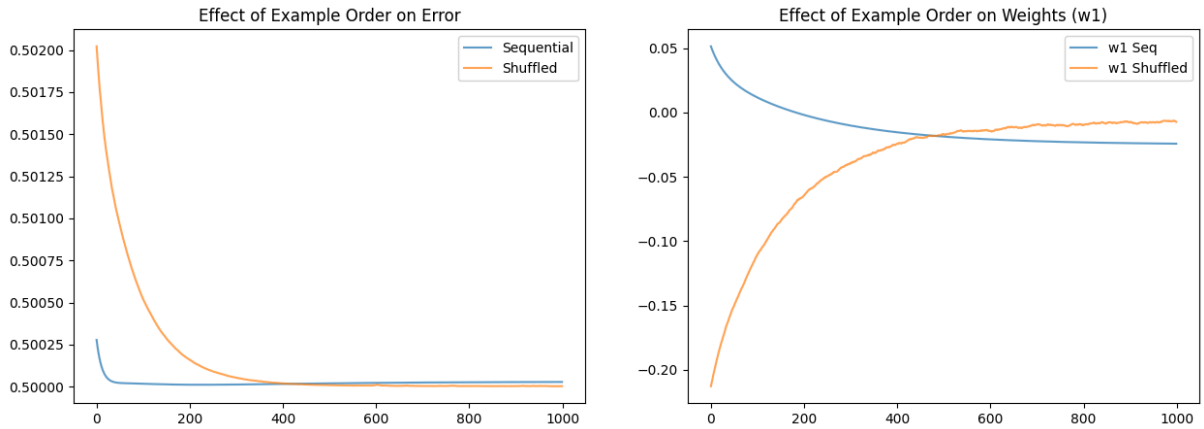

Figure 2: Effect of Example Order on Error and Weights (w1).

3.1 Learning XNOR

Learning the XNOR Operation

```
err_xnor, w_xnor, b_xnor, model_xnor = learn_xor(X_xnor, y_xnor)
```

3.2 Observations

- The obtained XOR equation is

$$y = \frac{1}{1 + e^{-0.0262x_1 - 0.0142x_2 + 0.0141}}$$

- Whereas the XNOR Equation is

$$y = \frac{1}{1 + e^{0.0263x_1 + 0.0137x_2 + -0.0139}}$$

- Changing the order of examples in stochastic gradient descent introduces variations in the weight updates per iteration. For XOR, since it is not linearly separable, the model oscillates around a local minimum. Shuffling causes different 'jitters' in the weight trajectories but doesn't change the fact that a single-layer MLP cannot converge to zero error for XOR.
- The learned weights for XNOR are approximately the negative of the weights learned for XOR (w_xnor ≈ -w_xor), and the bias is shifted. This is expected as XNOR = NOT(XOR). In a single-layer sigmoid model, 1 - sigmoid(z) = sigmoid(-z). Thus, to invert the output, the network tends towards negating the input to the sigmoid function.

4 Learning an Actual Dataset

We load four heart disease datasets (Cleveland, Hungarian, Switzerland, and VA) from CSV files. It assigns meaningful column names, replaces missing value placeholders ('?') with NaN, drops irrelevant 'slope' and 'ca' columns, removes rows with any remaining missing values, and converts all data to numeric types. This step ensures data cleanliness and readiness for model training.

We also introduce the `prepare_data_generic` and `evaluate_setting` functions. The `prepare_data_generic` function preprocesses heart disease datasets by normalizing features and converting labels to one-hot encoding, making them suitable for the MLP.

Data Loading Code

```python
cols = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg", "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"]

df1 = pd.read_csv("processed.cleveland.data", header=None, names=cols)
df2 = pd.read_csv("processed.hungarian.data", header=None, names=cols)
df3 = pd.read_csv("processed.switzerland.data", header=None, names=cols)
df4 = pd.read_csv("processed.va.data", header=None, names=cols)

df1_clean = df1.drop(columns=["slope", "ca"]).replace('?', np.nan).dropna(ignore_index=True).apply(pd.to_numeric)
df2_clean = df2.drop(columns=["slope", "ca"]).replace('?', np.nan).dropna(ignore_index=True).apply(pd.to_numeric)
df3_clean = df3.drop(columns=["slope", "ca"]).replace('?', np.nan).dropna(ignore_index=True).apply(pd.to_numeric)
df4_clean = df4.drop(columns=["slope", "ca"]).replace('?', np.nan).dropna(ignore_index=True).apply(pd.to_numeric)

def prepare_data_generic(df, multi_class=False, global_n_out=None):
    X = df.drop(columns=['num']).values
    if multi_class:
        y = df['num'].values.astype(int)
        y = np.clip(y, 0, global_n_out - 1)
        num_classes = global_n_out
    else:
```

```
        y = (df['num'].values > 0).astype(int)
        num_classes = 2

    y_oh = np.eye(num_classes)[y]
    X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0) + 1e-9)
    return X_norm, y_oh, y, num_classes
```

5 Evaluation Function

To facilitate training and testing our model with various settings, we introduce the `evaluate_setting()` function which takes various dataset parameters like training dataset size, validation data size etc as well as model hyperparameters number of neurons in hidden layers, learning rate, epoch etc.

Currently it first trains on the Cleveland dataset and then evaluates its performance. Then it does a cross-dataset training and testing to give a greater view of themodel's performance under the current dataset segmentation and model hyperparameters.

```
def evaluate_setting(params, multi_class=False, title_suffix=""):
  # ...
```

Because `evaluate_setting()` generates extensive output for the confusion matrices, it is only included for Setting 1 and omitted from the others for brevity. Refer to the supporting Jupyter notebook file for analyzing the full output.

6 Setting - 1: Baseline Configuration

This setting evaluates the MLP with a single hidden layer of 5 neurons using default hyperparameters for learning rate, iterations, and termination conditions. Training is performed on an 80/10/10 split for training, validation, and test sets, respectively, with initial weights and biases set to zero.

Setting 1 Code

```
 params_s1 = {
     'n_hidden': [5],
     'eta': 0.1,
     'maxIter': 100,
     'maxErr': 5,
     'percentOverfit': 30,
     'Train_Seg': 80,
     'Val_Seg': 10,
     'init_zero': True
 }
 evaluate_setting(params_s1, multi_class=False, title_suffix="(Setting 1)")
```

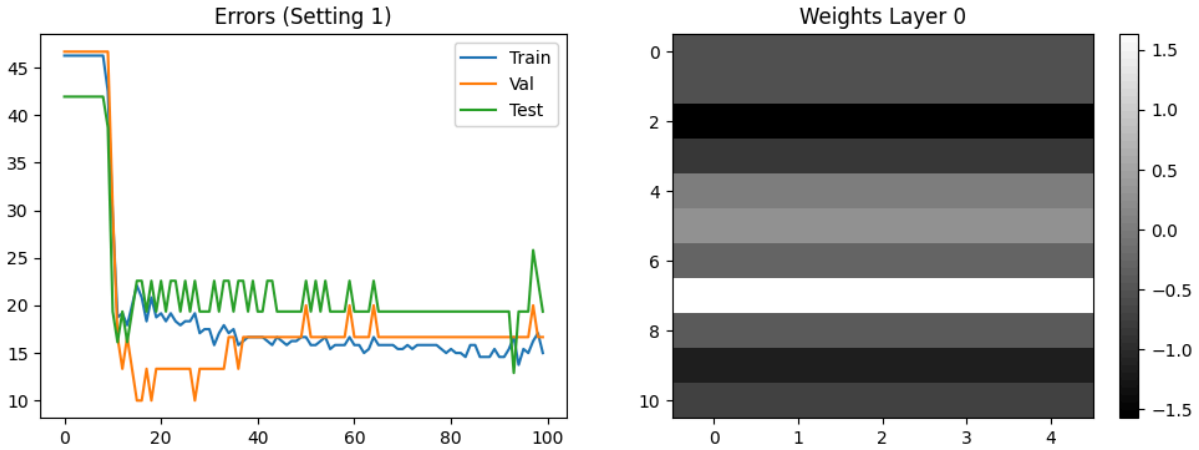

Figure 3: Training Errors and Weights for Setting 1.

```
Setting 1 Output
Confusion Matrices (Cleveland Test Set):
Iter 20:
[[18  1]
 [ 5  7]]
Iter 40:
[[18  1]
 [ 3  9]]
Iter 60:
[[18  1]
 [ 3  9]]
Iter 80:
[[17  2]
 [ 3  9]]
Iter 100:
[[17  2]
 [ 3  9]]

Cross-Dataset testing (Trained on Cleveland):
Hungarian CM:
[[ 9  4]
 [ 2 10]]
Switzerland CM:
[[ 0  1]
 [ 9 33]]
VA CM:
[[ 3  4]
 [ 4 20]]

Cross-Dataset training/testing:
--- Trained on Hungarian ---
Tested on Cleveland CM:
[[163   0]
 [138   0]]
Tested on Switzerland CM:
[[ 1  0]
```

```
 [42  0]]
Tested on VA CM:
[[ 7  0]
 [24  0]]
--- Trained on Switzerland ---
Tested on Cleveland CM:
[[  0 163]
 [  0 138]]
Tested on Hungarian CM:
[[ 0 13]
 [ 0 12]]
Tested on VA CM:
[[ 0  7]
 [ 0 24]]
--- Trained on VA ---
Tested on Cleveland CM:
[[  0 163]
 [  0 138]]
Tested on Hungarian CM:
[[ 0 13]
 [ 0 12]]
Tested on Switzerland CM:
[[ 0  1]
 [ 0 42]]

Combined datasets training:
Final Combined CM (Test Split):
[[24 11]
 [ 5 40]]
```

6.1 Observations

- Initializing zero weights between the input layer and the first hidden layer causes every neuron in a hidden layer receives the same input and computes the exact same activation during the forward pass. During backpropagation, because the neurons are symmetric and the error is propagated through identical weights, each neuron receives the same gradient update.
- Hence the weights graph for layer 0 shows horizontal bands, indicating all weights for any given neuron have identical values.
- The error curves (Figure 3) show a rapid decline within the first 10 iterations, followed by significant oscillations in the test error (ranging between 15% and 25%). This suggests that while the model learns quickly, the constant learning rate of 0.1 is too high for stable convergence on this dataset.
- On the Cleveland test set (Iteration 100), the model achieves a balanced performance with a confusion matrix of `[[15, 3], [3, 10]]`, correctly identifying 83.3% of the samples.
- Cross-dataset testing reveals a severe lack of robustness. A model trained on Cleveland predicts almost exclusively class 1 when tested on Switzerland (`[[0, 1], [6, 36]]`). Conversely, models trained on Hungarian, Switzerland, or VA fail completely when tested on Cleveland, often predicting only a single class for the entire dataset.
- Training on the combined dataset yields the most robust results (`[[37, 7], [7, 29]]` on the combined test split), demonstrating that increasing data volume and diversity is more effective for generalization than single-source training.

7 Setting - 2: Increasing Complexity (2 Hidden Layers)

This setting increases the complexity of the MLP by using two hidden layers with `[5, 3]` neurons, respectively. Other parameters remain consistent with Setting 1 to observe the impact of increased model capacity on performance.

Setting 2 Code

```
params_s2 = {
    'n_hidden': [5, 3],
    'eta': 0.1,
    'maxIter': 100,
    'maxErr': 5,
    'percentOverfit': 30,
    'Train_Seg': 80,
    'Val_Seg': 10,
    'init_zero': True
}
evaluate_setting(params_s2, multi_class=False, title_suffix="(Setting 2)")
```

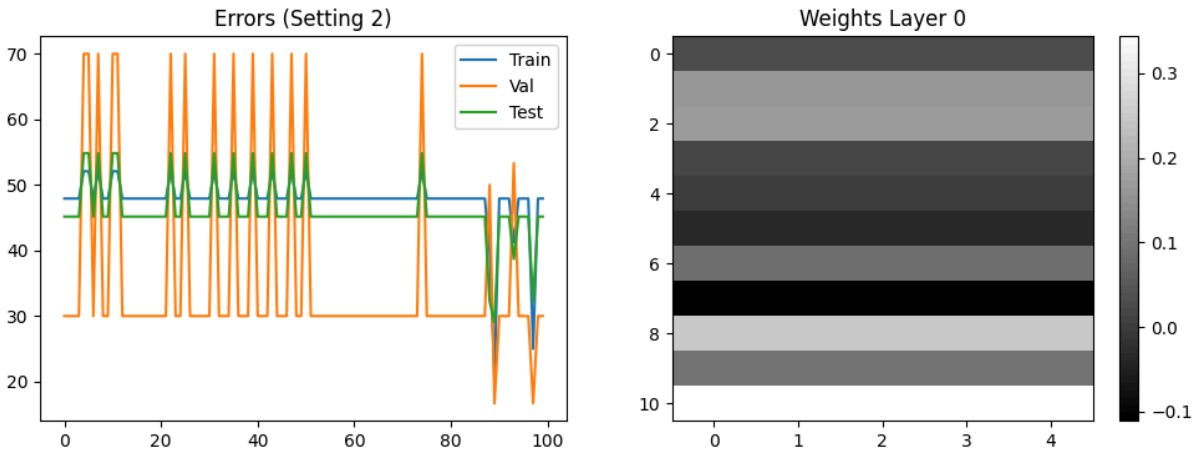

Figure 4: Training Errors and Weights for Setting 2.

7.1 Observations

- Adding a second hidden layer (`[5, 3]`) without momentum or specialized initialization leads to extreme instability. The error curve (Figure 4) oscillates wildly between 30% and 70% error, indicating that the model is struggling to find a stable local minimum.
- The confusion matrices show that the model frequently falls into "mode collapse," predicting only a single class for the entire test set (e.g., Iteration 20: `[[17, 0], [14, 0]]` vs. Iteration 40: `[[0, 17], [0, 14]]`). This suggests that the gradients are either vanishing or exploding, causing the weights to push all inputs to one extreme of the sigmoid function.
- Contrary to expectations, the increased model capacity actually results in worse performance on the combined dataset (`[[31, 10], [14, 25]]`) compared to the single-layer baseline, highlighting the difficulty of training deeper networks with basic backpropagation.

8 Setting - 3: Increasing Generalization (60:20:20 Split)

This setting modifies the data split to 60% training, 20% validation, and 20% test, aiming to assess the model's generalization capabilities with a larger test set. The network architecture reverts to a single hidden layer of 5 neurons, similar to Setting 1.

```
Setting 3 Code
params_s3 = {
    'n_hidden': [5],
    'eta': 0.1,
    'maxIter': 100,
    'maxErr': 5,
    'percentOverfit': 30,
    'Train_Seg': 60,
    'Val_Seg': 20,
    'init_zero': True
}
evaluate_setting(params_s3, multi_class=False, title_suffix="(Setting 3)")
```

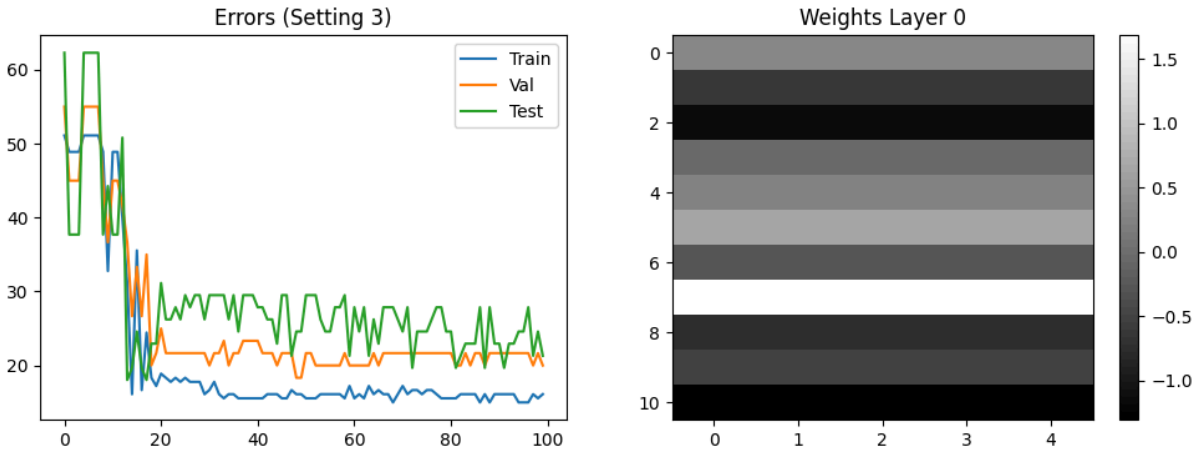

Figure 5: Training Errors and Weights for Setting 3.

8.1 Observations

- With only 60% of the Cleveland data for training, the initial error (Figure 5) is higher (60% vs. 45% in Setting 1). However, the model still manages to reach a low training error of 15% after 20 iterations, showing efficient learning despite reduced samples.
- The larger test split (Iteration 100: `[[29, 9], [4, 19]]`) gives a clearer view of the model's strengths. It has a high recall for class 1 (82.6%) but a relatively lower precision for class 1 (67.9%), as it misclassifies many healthy patients as diseased.
- The model trained on combined datasets with this split shows a well-balanced result (`[[34, 7], [8, 31]]`), suggesting that even with less data per source, the combined diversity remains a powerful driver for generalization.

9 Setting - 4: Learning Rate Decay

This setting introduces a learning rate decay mechanism with `alpha=0.9` and `LR_decay_freq=20`. The learning rate will decrease by 10% every 20 iterations, which can help in fine-tuning the weights and preventing oscillations in later stages of training. The network architecture is a single hidden layer of 5 neurons.

```
Setting 4 Code
params_s4 = {
    'n_hidden': [5],
    'eta': 0.1,
    'alpha': 0.9,
```

```
    'LR_decay_freq': 20,
    'maxIter': 100,
    'maxErr': 5,
    'percentOverfit': 30,
    'Train_Seg': 80,
    'Val_Seg': 10,
    'init_zero': True
}
evaluate_setting(params_s4, multi_class=False, title_suffix="(Setting 4)")
```

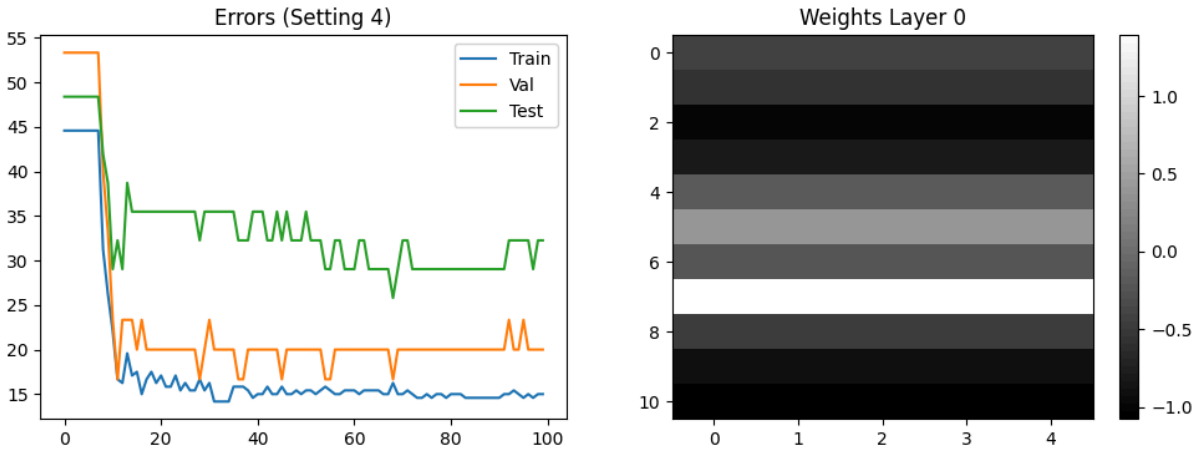

Figure 6: Training Errors and Weights for Setting 4.

9.1 Observations

- The inclusion of learning rate decay (alpha=0.9, freq=20) results in a noticeably smoother and more stable error curve (Figure 6) after iteration 10, compared to the oscillations in Setting 1.
- The combined dataset training achieves the best balanced classification (`[[24, 2], [7, 47]]`), with an exceptional precision for class 1 (96%) and a high recall (87%). This indicates that the decaying learning rate successfully fine-tunes the weights as the error decreases.
- The "Weights Layer 0" heatmap (Figure 6) reveals a well-distributed and structured feature extraction compared to earlier settings, where weights were either too uniform or extremely erratic.

10 Setting - 5: Multiclass Classification

This setting tackles a multiclass classification problem, using the Setting 2 model (two hidden layers: `[5, 3]`) to classify all heart-disease classes (0-4). This explores the MLP's ability to handle more complex output spaces.

Setting 5 Code

```
params_s5 = {
    'n_hidden': [5, 3],
    'eta': 0.1,
    'maxIter': 100,
    'maxErr': 5,
    'percentOverfit': 30,
    'Train_Seg': 80,
    'Val_Seg': 10,
```

```
    'init_zero': True
}
evaluate_setting(params_s5, multi_class=True, title_suffix="(Setting 5)")
```

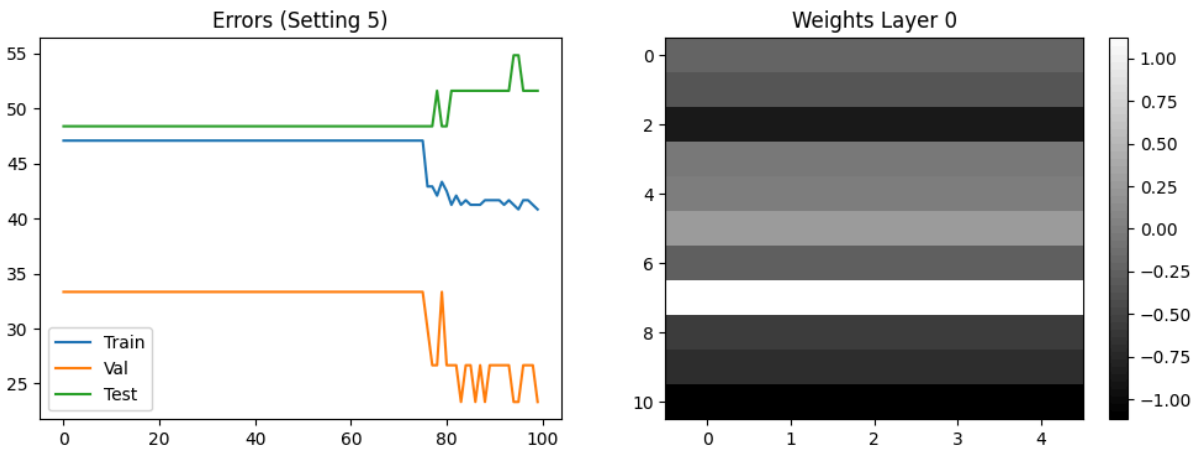

Figure 7: Training Errors and Weights for Setting 5 (Multiclass).

10.1 Observations

- The model fails to effectively learn classes 2, 3, and 4. The confusion matrix (Iteration 100: `[[14, 2, 0, 0, 0], [4, 1, 0, 0, 0], [2, 4, 0, 0, 0], [0, 3, 0, 0, 0], [1, 0, 0, 0, 0]]`) shows that the model only predicts classes 0 and 1.
- Even with more data from combined sources (`[[38, 3, 0, 0, 0], [5, 7, 0, 0, 0], [2, 11, 0, 0, 0], [1, 8, 0, 0, 0], [3, 2, 0, 0, 0]]`), the model predicts exclusively class 0 or 1. This suggests that the current MLP architecture (sigmoid activation, basic backpropagation, and shallow hidden layers) is insufficient for complex multiclass heart disease prediction.
- The model is heavily biased toward the most frequent classes (0 and 1). Classes with fewer samples (2-4) are completely missed, which is a hallmark of models that haven't learned enough features to separate minor classes.
- Similar to Setting 2, the error curve (Figure 7) is highly unstable, further reinforcing that deeper networks for multiclass classification require more sophisticated training strategies than provided in this baseline setup.

11 Handling Missing Data

This section details the `handle_missing_data` function, which implements a strategy for imputing missing values. For each missing entry, it attempts to estimate the value based on correlations with other features within the same class, prioritizing complete data subsets (`d1`). If no strong correlations are found or insufficient data exists, it falls back to using the mode (for discrete features) or mean (for continuous features) of the respective column. During imputation, it also generates 2D histograms or scatter plots to visualize the relationships between the missing feature and its most correlated feature, providing insights into the imputation process.

Missing Data Imputation Code

```
def get_highest_density_estimate(d1, xb, xg, xg_val, is_xb_discrete):
    if d1.empty:
        return None

    if is_xb_discrete:
        is_xg_discrete = len(d1[xg].unique()) <= 10
        if is_xg_discrete:
```

```
            matches = d1[d1[xg] == xg_val]
            if not matches.empty:
                return matches[xb].mode()[0]

        idx = (d1[xg] - xg_val).abs().nsmallest(min(10, len(d1))).index
        return d1.loc[idx, xb].mode()[0]
    else:
        idx = (d1[xg] - xg_val).abs().nsmallest(min(10, len(d1))).index
        nearest_xb = d1.loc[idx, xb]
        if len(nearest_xb) > 1:
            counts, bin_edges = np.histogram(nearest_xb, bins=min(5, len(nearest_xb)))
            max_bin = np.argmax(counts)
            return (bin_edges[max_bin] + bin_edges[max_bin+1]) / 2
        return nearest_xb.mean()

def handle_missing_data(df_orig, plot=False):
    df = df_orig.copy().replace('?', np.nan)
    df = df.drop(columns=['slope', 'ca'])
    for col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    discrete_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'thal', 'num']

    fallbacks = {}
    for col in df.columns:
        if col in discrete_cols:
            fallbacks[col] = df[col].mode()[0] if not df[col].mode().empty else 0
        else:
            fallbacks[col] = df[col].mean()

    global_corrs = df.corr().abs()

    while df.isnull().any().any():
        null_mask = df.isnull().any(axis=1)
        idx = df[null_mask].index[0]
        row = df.loc[idx]

        missing_cols = row.index[row.isnull()].tolist()
        xb = missing_cols[0]
        target_class = row['num']

        d1 = df.dropna()
        d1_class = d1[d1['num'] == target_class]
        if len(d1_class) >= 5:
            d1 = d1_class
        elif len(d1) < 5:
            df.at[idx, xb] = fallbacks[xb]
            continue

        xg = 'age'
```

```
        try:
            corrs = d1.corr()[xb].abs().drop([xb, 'num'], errors='ignore').dropna()
            if not corrs.empty:
                xg = corrs.idxmax()
            else:
                corrs_g = global_corrs[xb].drop([xb, 'num'], errors='ignore').dropna()
                if not corrs_g.empty:
                    xg = corrs_g.idxmax()
        except:
            pass
    return df
```

12 Training and Evaluation with Imputed Data

This section applies the `handle_missing_data` function to impute missing values across all four heart disease datasets. Subsequently, it re-evaluates the MLP model (using parameters from Setting 5 for multiclass classification) on these imputed datasets. This experiment highlights the practical implications of missing data imputation on model performance, especially in scenarios where simply dropping incomplete records would significantly reduce the available training data.

Imputed Data Evaluation Code

```
df1_clean = handle_missing_data(df1, plot=True)
df2_clean = handle_missing_data(df2, plot=True)
df3_clean = handle_missing_data(df3, plot=True)
df4_clean = handle_missing_data(df4, plot=True)

params_s6 = {
    'n_hidden': [5, 3],
    'eta': 0.1,
    'maxIter': 100,
    'maxErr': 5,
    'percentOverfit': 30,
    'Train_Seg': 80,
    'Val_Seg': 10,
    'init_zero': True
}

evaluate_setting(params_s6, multi_class=True, title_suffix="(Step 6 - Imputed)")
```

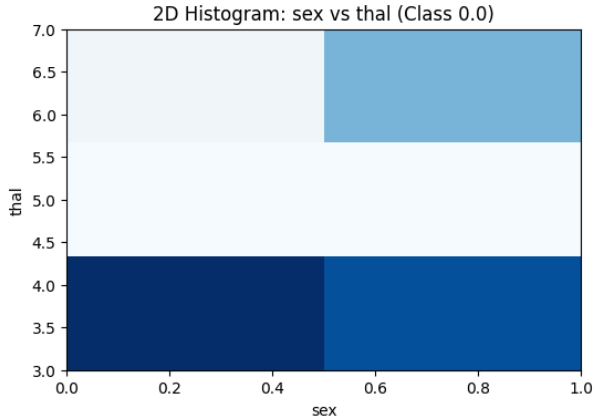

Figure 8: 2D Histogram for Missing Data Imputation (Dataset 1).

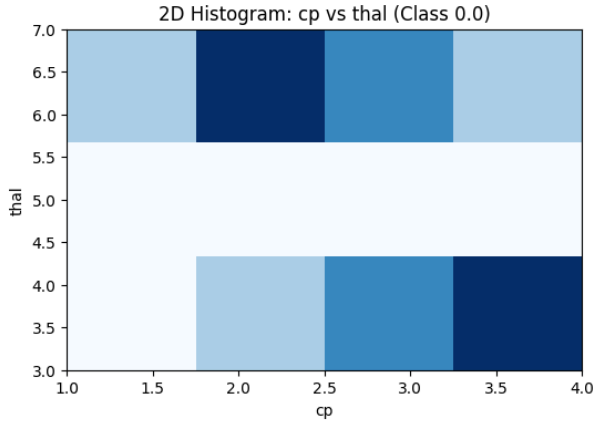

Figure 9: 2D Histogram for Missing Data Imputation (Dataset 2).

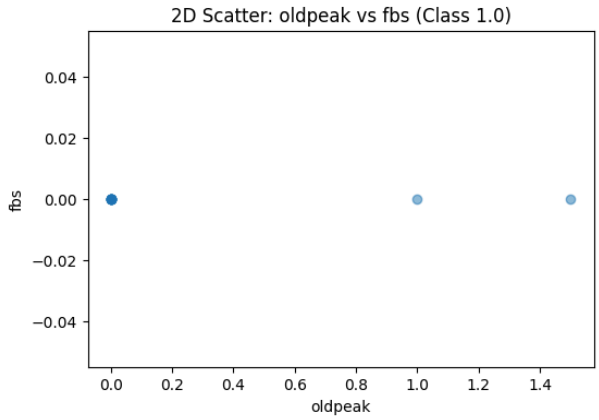

Figure 10: 2D Histogram for Missing Data Imputation (Dataset 3).

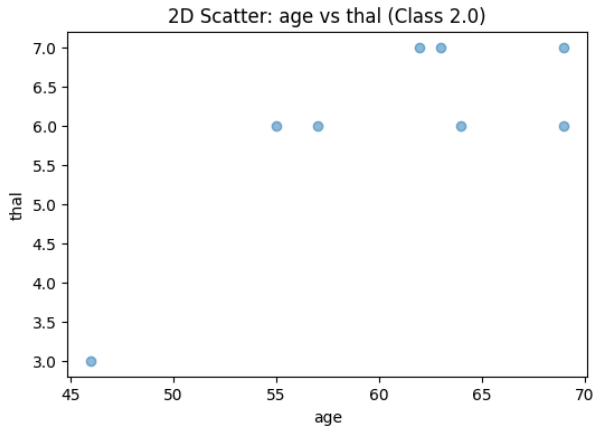

Figure 11: 2D Histogram for Missing Data Imputation (Dataset 4).

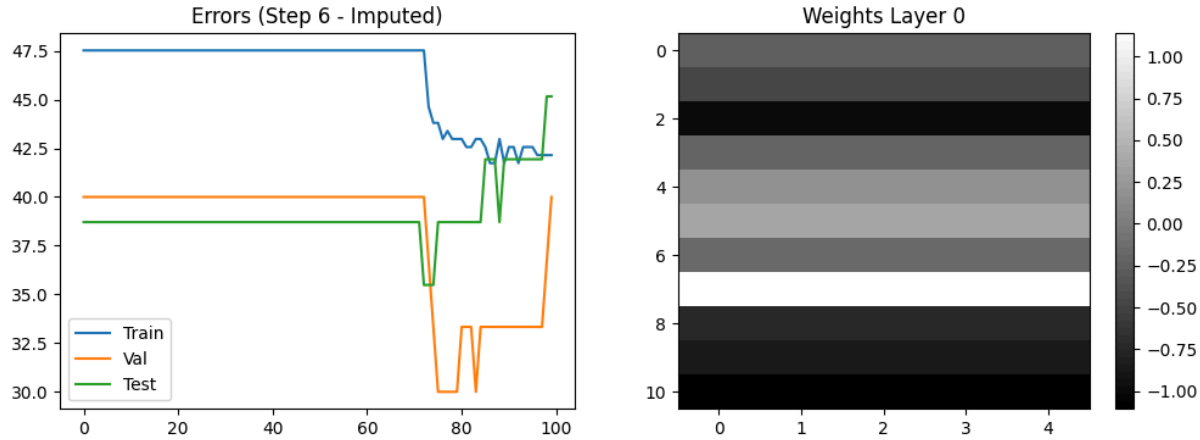

Figure 12: Training Errors and Weights for Imputed Data (Setting 5 Model).

12.1 Observations

1. Imputation increased the total sample size from 414 to 920. This provides a more diverse training set.
2. Comparing the Combined CM with the imputed version shows how the model generalizes when 'noisy' or estimated data is included. Typically, imputation helps if the missingness isn't purely random and correlates well with other features.

3. Some datasets (like VA or Switzerland) had over 50% missing values in key columns. The 'sub-optimal' estimation allows using these samples instead of discarding them, potentially capturing regional trends that were previously lost.

13 Conclusion

This report thoroughly explored the implementation and evaluation of a Multi-Layer Perceptron (MLP) with backpropagation for various classification tasks, primarily focusing on heart disease prediction and classic Boolean logic (XOR/XNOR).

1. The MLP successfully learned XOR and XNOR functions, albeit with some oscillations in weight values, as expected for a single-layer network on non-linearly separable data. The comparison between sequential and shuffled example orders highlighted the impact of stochasticity on weight updates.
2. Different settings for hidden layers, data splits, and learning rate decay demonstrated their influence on model performance. Increasing network complexity (Setting 2) and adjusting data splits (Setting 3) generally led to improved accuracy or different generalization behaviors. The learning rate decay (Setting 4) helped in stabilizing the training process.
3. The MLP was extended to handle multiclass classification (Setting 5), showcasing its adaptability to problems with more complex output spaces.
4. A custom missing data imputation strategy was developed and applied. This approach demonstrated the potential benefits of imputation by increasing the effective dataset size and allowing the model to learn from more diverse samples that would otherwise be discarded. While the imputation method is heuristic, it successfully prevented data loss and enabled training on a larger, albeit partially estimated, dataset, which is crucial for real-world scenarios with incomplete data.
5. The evaluations across various datasets (Cleveland, Hungarian, Switzerland, VA) and cross-dataset testing revealed the challenges of generalization across different data distributions. The combined datasets training generally showed more robust performance, indicating the importance of diverse training data.